

Appunti di Programmazione

UNIPD - Fabio Aioli

FILIPPO GARGIULO

Ottobre - Novembre 2025

Indice

1	Programmazione orientata agli oggetti	2
1.1	referimenti	3
1.2	Input e Output	3
1.3	tipi	4
2	Tipi	4
2.1	Dizionari	4
2.2	Insiemi	5

§1 Programmazione orientata agli oggetti

Python è un linguaggio orientato agli oggetti. Cos'è un oggetto? Prima di tutto è un contenitore di informazioni, ovvero un modo per immagazzinare dati di varia natura in vari modi. In python tutto il linguaggio si basa sulla relazione tra diversi oggetti.

Un oggetto in python ha tre diverse caratteristiche:

Identità: è un codice che identifica ciascun oggetto, non c'entra nulla con il suo indirizzo di memoria. Per visualizzare l'identità di un oggetto in python possiamo usare la funzione

```
1 id(Obj)
2
```

Due oggetti hanno la stessa identità se sono lo stesso oggetto. Anche per capire ciò esiste un comando python

```
1 Obj1 is Obj2
2
```

Questo ritorna un bool.

Tipo: Riguarda tutte le operazioni che si possono fare con quel particolare oggetto. In python i tipi presenti sono fondamentalmente numerici, e stringa. Ci sono anche altri tipi più avanzati. Per vedere il tipo di un oggetto bisogna scrivere:

```
1 type(Obj)
2
```

Il tipo caratterizza il genere di operazioni che si possono effettivamente fare con quell'oggetto.

Valore: Sono i dati effettivamente immagazzinati nell'oggetto. Per verificare se due oggetti hanno effettivamente lo stesso valore usiamo

```
1 Obj1==Obj2
2
```

anche questo ritorna un bool.

Gli oggetti possono essere combinati tramite degli operatori. Python sulla base del tipo assegna comportamenti diversi ad operatori uguali. Per esempio

```
1 "pinco"+"pallino"
2
```

restituisce la stringa "pincopallino", questo si chiama **overloading** degli operatori. Anche `*` è un operatore che funziona con overloading su stringhe.

Gli oggetti possono anche essere usati come input/output di funzioni

```
1 len("abecedario")
2
```

restituisce la "lunghezza" dell'oggetto. Cosa ciò voglia dire dipende dall'overload. Un particolare tipo di funzione è il metodo, ovvero una funzione che è associata ad una particolare classe.

```
1 "abecedario".count("ec")
2
```

conta il numero di volte che la stringa "ec" occorre nella stringa.

§1.1 riferimenti

I riferimenti sono essenzialmente degli alias per gli oggetti (praticamente sono delle variabili):

```
1 x = "barabba"
2
```

In questo modo non stiamo creando un nuovo oggetto, ma stiamo solo indicando un modo abbreviato per chiamarlo. Infatti l'identità di un oggetto è la stessa di tutti i suoi riferimenti.

```
1 x = "blu"
2 y = "verde"
3 z = x
4
```

`z` è un nuovo riferimento, ma non si riferisce a `x`. Infatti in python riferimenti non possono mai riferirsi ad altri riferimenti. `z` si riferisce direttamente al valore a cui si riferisce `x`, ovvero `"blu"`.

Non tutte le stringhe possono essere usate come nomi per riferimenti:

0. In generale un riferimento dovrebbe rispecchiare l'oggetto a cui si riferisce.
1. Il primo carattere deve essere una lettera maiuscola o minuscola, o un `_`.
2. Si possono usare solo caratteri alfanumerici o underscore (niente caratteri di interpunzione).
3. Niente parole chiave del linguaggio.

Python supporta l'assegnamento in parallelo:

```
1 x,y=1,2
2
```

e (come in C) supporta operazioni del tipo

```
1 x=23
2 x+=3 #equivalente a x=x+3
3
```

Anche qui non è l'oggetto a cui si riferiva `x` a cambiare, ma è `x` che smette di riferirsi a 23 per riferirsi a 26.

§1.2 Input e Output

Per prendere input e output abbiamo diverse comode funzioni in Python. La funzione `print` stampa su schermo il valore dell'oggetto che gli si dà in input, e returna come valore `NaN`. Python in realtà stampa di default il valore di return dell'ultima espressione del programma, oltre ai valori che gli sono richiesti di printare tramite `print`.

La funzione `input(Obj)` invece stampa a schermo il valore di `Obj`, e poi si mette in attesa di un input dall'utente. Infine returna quanto scritto dall'utente. Per esempio

```
1 nome = input("come ti chiami")
2 print("ciao", nome)
3
```

stamperà `ciao <nome>`.

§1.3 tipi

esistono un sacco di tipi, le categorie principali sono

- Numerici (`int`, `float`, `complex`, `bool`)
- Stringhe (`string`)
- Tuple, contenitori di oggetti eterogenei (`tuple`).
- Liste, vettori C-style (`list`).
- Dizionari, liste di pair eterogenee (`dict`).
- Insiemi (`set`).

§2 Tipi

§2.1 Dizionari

I dizionari sono strutture dati che ci permettono di memorizzare "informazioni associative", ovvero di associare a ogni dato un altro dato, di tipo possibilmente arbitrario. Non ci possono essere due chiavi uguali in un dizionario.

Per inizializzare un dizionario possiamo usare sia la notazione con le graffe che il costruttore

```
1 primo_vuoto = {}
2 secondo_vuoto = dict()
3 # entrambe queste notazioni restituiscono un dizionario vuoto
4
5 students = {'2199095': ('Filippo Gargiulo', '19/11/2006')}
6 # questo crea un dizionario con gli oggetti specificati sopra
7
```

possiamo anche utilizzare la notazione con le quadre per inserire elementi nel dizionario

```
1 students['2199095'] = ('Filippo Gargiulo', '19/11/2006')
2
```

o per accedervi, infatti

```
1 print(students['2199095'])
2
```

stampa `('Filippo Gargiulo', '19/11/2006')`. Con la stessa notazione è anche possibile modificare il valore associato ad una certa chiave. Tutti i tipi immutabili sono utilizzabili come chiave (non le liste, purtroppo), inoltre se un container è usato come chiave anche tutti gli oggetti che contiene devono essere immutabili. Questo è legato al fatto che i dizionari lavorano con gli hash, e dunque ha bisogno di tipi immutabili per non far mutare gli hash.

In effetti il fatto che i dict funzionino con gli hash permette di avere check estremamente efficienti del fatto che una chiave appartenga o non appartenga ad un certo dict.

alcuni metodi utili

```
1 d = {'K1': 1, 'K2': 2}
2 d.keys()
3 d.values()
4 d.items()
5
```

che restituiscono delle liste (in realtà ciascuno di essi è un tipo particolare, ma è tutto convertibile a lista senza problemi) con gli oggetti delle chiavi, dei valori o le coppia chiave-valore.

§2.2 Insiemi

I set sono fondamentalmente diversi perchè non possono avere duplicati dello stesso oggetto. La maggior parte delle cose che si possono fare algebricamente con gli insiemi matematici si possono fare con gli insiemi di python.

```
1 pari = set(2*k in range(1,20))
2 disp = set(1+2*k in range(1,20))
3
4 unione = disp | pari
5 intersezione = disp & pari
6 differenza = pari - disp
7 differenza_simmetrica = disp ^ pari
8
9 #aggiungere e rimuovere elementi si fa facilmente con
10
11 pari.add(434)
12 disp.remove(3)
13
```